

Meme Creator: Case Study 1 Report

- Space: <https://huggingface.co/spaces/minh-hahaha/CaseStudy1>
- GitHub: <https://github.com/minh-hahaha/CaseStudy1>
- Demo: <https://youtu.be/CZx28BOT-jk>
- [CITATIONS.md](#)

a. Members

- Minh Ha
- Max Jeronimo

b. Product description

Meme Creator turns a photo into a captioned meme: upload an image, pick a voice (*Professor Energy*, *Dad Joke*, *Corporate LinkedIn*), optionally add a topic, and the app renders the chosen caption in classic top/bottom meme style.

Purpose: a fast, workplace-safe meme without writing the joke yourself.

Audience: professors, managers, and presenters opening a talk with a meme, and students making slides.

c. Models

Model	Role	Runs
Salesforce/blip-image-captioning-base	Describes the image in one sentence	Locally, both paths
openai/gpt-oss-20b	Writes captions from that description	Remote, Inference API
Qwen/Qwen3-0.6B	Writes captions from that description	Locally on the Space

d. Performance analysis

Metric	Remote (gpt-oss-20b)	Local (Qwen3-0.6B)
BLIP vision time (shared step)	6.71s	6.71s
Caption generation time	0.64s	8.03s
Cold start (first request)	0.31s	11.94s
Host resources	Network only	ZeroGPU slice, ~1.4 GB model

Averages over 3 images x 4 runs (1 cold + 3 warm) from the Model Lab tab. The remote model was faster (<1s warm) and produced funnier, more on-style captions than local Qwen3-0.6B.

e. Cost analysis (1,000 users)

LLM-assisted estimate; pricing sources cited. Assumption: 1,000 users x 5 req/day $\approx$ 150,000 req/month, $\sim$ 300 input + $\sim$ 400 output tokens/request ([pricing](#)).

- **Shared BLIP cost** (both paths, every request): $6.71s \times 150,000 \approx 280$ GPU-hrs/month on a T4 (\$0.40/hr) $\approx$ **\$112/month**.
- **Remote path:** gpt-oss-20b at \$0.10/\$0.50 per 1M in/out tokens $\approx$ \$0.00023/request $\approx$ \$35/month in tokens + \$112 BLIP $\approx$ **\$147/month total**. Free/PRO inference credits (\$0.10/\$2.00) are exhausted in 1-9 requests, so this runs on pay-as-you-go billing.
- **Local path:** free ZeroGPU quota can't cover 1,000 daily users; a dedicated T4 running continuously (BLIP + Qwen3, $\sim$ 613 GPU-hrs fits in $\sim$ 730 hrs/month) costs $24 \times 30 \times \$0.40 \approx$ **\$288/month**, billed by uptime regardless of actual traffic.
- **Summary:** remote ($\sim$ \$147/mo) beats a continuously-running dedicated GPU ($\sim$ \$288/mo) at this scale; crossover is $\approx$ 765,000 requests/month, above which the flat-rate GPU wins.

f. Comments and concerns

- **Privacy:** Remote mode sends the image *description* (not the image) to a third party; Local-only mode keeps everything on the Space.
- **Reliability:** the remote path can fail (rate limits, outages, timeouts, empty response); failover to Qwen3 keeps the app up with lower quality.
- **Content safety:** the prompt forbids slurs/sexual content, but there is no output filter.

g. Improvement suggestion from an established LLM [LLM only]

Model: Claude Sonnet 5

Prompt:

Meme Creator is a Gradio app on Hugging Face Spaces: a user uploads a photo, a local BLIP model describes it, and captions are then written either by a remote LLM (openai/gpt-oss-20b, via the Hugging Face Inference API) or, on failure, a local LLM (Qwen/Qwen3-0.6B) running on the same host. Identify one meaningful improvement related to performance, reliability, scalability, cost, security, or maintainability.

Response:

One meaningful improvement is adding request-level rate limiting around the remote path, specifically for anonymous visitors who fall back to the Space's own HF_TOKEN. In `remote_llm.py`, `candidate_tokens()` silently uses the Space owner's token whenever a viewer has no OAuth grant (or their grant lacks the inference scope), and `router.make_captions` places no bound on how many times any single visitor can trigger that path per minute. A scripted or malicious visitor could drive the Space owner's pay-as-you-go bill up quickly, or exhaust the account's request quota, without ever authenticating. I'd add a lightweight per-session (or per-IP) token-bucket limiter — e.g., N requests per minute keyed on the Gradio session hash — that returns a "please slow down" message before the request reaches `remote_llm.generate_captions`. This is cheap to add (a dict of timestamps or `cachetools.TTLCache`), needs no new infrastructure, and protects both cost (the estimate in item e assumes well-behaved traffic) and availability, since an abusive burst would also degrade latency for legitimate users sharing the same fallback token.

h. Same prompt, weaker local LLM [LLM only]

Model: Qwen/Qwen3-0.6B

Response:

One meaningful improvement related to performance, reliability, scalability, cost, security, or maintainability is **increasing the model's inference speed**.

Why this improvement is relevant: - **Speed:** The Hugging Face Inference API typically has limitations on latency, especially for large models like GPT-3 or Qwen. By optimizing the model to run faster, the system can handle more requests per second, improving overall performance and user experience. -

Scalability: With faster inference, the system can process more data and respond to more queries, which is crucial for real-time applications. - **Cost:** If the model is optimized to use fewer resources (e.g., a more efficient inference architecture), the cost could be reduced.

This improvement enhances the user experience and supports larger-scale deployments.

i. Comparison of the two responses [Human only]

Claude's response was more useful, accurate and actionable. It named specific functions and identified concrete risk. Qwen was vague and largely generic. It referenced GPT-3 but that model was not part of this stack. It also didn't propose any actionable insights

j. Model choice for real deployment [Human only]

We'd choose gpt-oss-20b as the primary path as it is faster and cheaper and produced better captions in testing. Local Qwen could be used as a failover to protect against the main weakness of the remote path if outage ever to happened.

k. Adaptive LLM failover (Extra Credit)

Advantages: the app stays up through rate limits, outages, or timeouts, requires no user intervention.

Disadvantages: users may not notice why quality dropped, the failure path pays both a remote timeout and local generation time, and each new request retries remote from scratch instead of remembering the last failure.